

MOM (Minutes of Meeting) System Documentation

Index / Table of Contents

1. Project Overview & Flow
2. File Structure
3. MOM (Minutes of Meeting) Module Details
4. Database Architecture (Models)
5. API Integrations (Master API, Calendars, SMTP)
6. Security & Authentication

1. Project Overview & Flow

The **Minutes of Meeting (MOM) System** is a full-stack web application built on the MERN stack (MongoDB, Express, React, Node.js). Its primary purpose is to help organizations schedule meetings, invite members, generate MOM documents (collaboratively in real-time), assign tasks (Action Items), and send automated email summaries.

Complete Project Flow:

1. **User Authentication:** Users register and login via JWT. The system also supports Third-Party API Key authentication for external integrations (Master API).
2. **Workspace Management:** Users belong to Workspaces. Data is isolated per workspace ensuring multi-tenant security.
3. **Meeting Creation:** An admin/employee schedules a meeting, invites guests (internal/external), and sets the agenda. Integration with Google/Microsoft Calendars allows syncing.
4. **MOM Editor (Real-Time):** During the meeting, participants use a rich-text collaborative editor (Yjs / Hocuspocus) to take notes, add action items, and make decisions.
5. **Action Items (Tasks):** Tasks are assigned to specific users with deadlines. The system runs automated background cron jobs (Reminders) to notify users of pending tasks.
6. **Finalization & Export:** Once the meeting ends, the MOM is finalized. It can be exported as a PDF and shared via email or public link.

2. File Structure

The project follows a standard decoupled Client-Server architecture.

```
MOM-PROJECT/
├── client/                                # React Frontend
│   ├── src/
│   │   ├── api/                         # Axios API instances (API Versioning v1)
│   │   ├── components/                 # Reusable UI components (Modals, Cards, etc.)
│   │   ├── hooks/                     # Custom React hooks (useAuth, useWorkspace)
│   │   ├── layouts/                   # Page layouts (Sidebar, Header)
│   │   ├── pages/                     # Individual pages (Dashboard, Settings, MOM Editor)
│   │   └── utils/                     # Helpers and formatters
│   └── package.json                    # Frontend Dependencies (Vite, Tailwind, React)
├── server/                              # Node.js / Express Backend
│   ├── config/                         # Environment configs
│   ├── controllers/                   # Business logic for routes
│   ├── middlewares/                  # Auth (JWT & API Key), Validation, Workspaces
│   ├── models/                       # Mongoose Schemas
│   ├── routes/                       # Express routes mapped to /api/v1/
│   ├── services/                     # External API & Background services (SMTP, Collab)
│   └── uploads/                      # Static files and attachments
```

```
| └─ server.js      # Main application entry point
```

3. MOM (Minutes of Meeting) Module Details

The core of the application revolves around the Minutes of Meeting functionality.

- **Collaborative Editing:** Powered by `Yjs` and `Hocuspocus` WebSockets, allowing multiple users to type simultaneously in real-time.
- **Action Items Tracking:** Within the MOM, decisions are converted into Action Items (Tasks). These are tracked independently with statuses (Pending, In Progress, Completed).
- **Attachments:** Users can upload relevant files (PDFs, Images) to a meeting.
- **Sharing & Visibility:** MOMs can be shared via a secure, tokenized public link for external visitors.
- **PDF Generation:** Utilizes `Puppeteer` on the backend to dynamically render the MOM HTML content into a downloadable PDF document.

4. Database Architecture (Models)

The MongoDB database contains several interconnected collections:

Model	Purpose
<code>User</code>	Stores user credentials, name, and email.
<code>Workspace</code> & <code>Membership</code>	Handles multi-tenancy. Connects Users to Workspaces with Roles (Admin, Employee).
<code>Meeting</code>	Stores meeting schedules, title, agenda, and participants.
<code>Mom</code>	Stores the actual Minutes text, HTML content, and links to the Meeting.
<code>ActionItem</code>	Stores assigned tasks, deadlines, and current status.
<code>ApiKey</code>	Stores hashed Developer API Keys for third-party Master API integrations.
<code>IntegrationToken</code>	Stores OAuth tokens for Google/Microsoft Calendar integrations.

5. API Integrations (Master API, Calendars, SMTP)

The Master API (v1)

The entire backend functions as a **Master API**. All routes are prefixed with `/api/v1/`. External systems (like HRMS or Mobile Apps) can interact with this API by sending a generated `x-api-key` in the header. The system automatically identifies the Workspace and User associated with the key.

Third-Party Integrations

- **Google Calendar API:** Used via OAuth 2.0 to sync meetings scheduled in the MOM system directly to the user's Google Calendar.
- **Microsoft 365 (Outlook):** Used via Microsoft Graph API for scheduling and syncing meetings with Outlook accounts.
- **SMTP (Nodemailer):** The system is configured to send emails using standard SMTP. This handles meeting invitations, task assignment alerts, and daily reminders.
- **Generative AI (Gemini/OpenAI):** Optional AI integration used in the `aiRoutes.js` to summarize meeting transcripts or generate templates automatically.

6. Security & Authentication

The application implements a robust dual-authentication hybrid system:

1. **Web Client (JWT):** The React app uses JSON Web Tokens stored securely on the client. Every request sends an `Authorization: Bearer <token>` header.
2. **Third-Party Apps (API Key):** External apps use an API Key generated from the Settings dashboard. They send an `x-api-key: <key>` header. The server hashes the incoming key using `SHA-256` to verify it against the database securely.
3. **Workspace Isolation:** The `workspaceMiddleware.js` strictly validates that a user or API key has explicit permissions (Membership) to access or modify resources within a specific Workspace context.